Sentinel Lakehouse: Gold Accelerators Architectural Roadmap

Target Architecture: Databricks Medallion (Bronze → Silver → Gold) **Core Focus:** Zero-Overhead DQ, Late-Arrival Auto-Reconciliation, Alert Fatigue

Architectural Roadmap: Sentinel Lakehouse Plug-and-Play Gold Accelerators

1. Executive Vision & Problem Statement

The Databricks Cost Paradox

Modern enterprise data platforms operating on Databricks follow the Medallion Architecture (Bronze -> Silver -> Gold). When scaling to production, teams face critical operational challenges at the Gold layer:

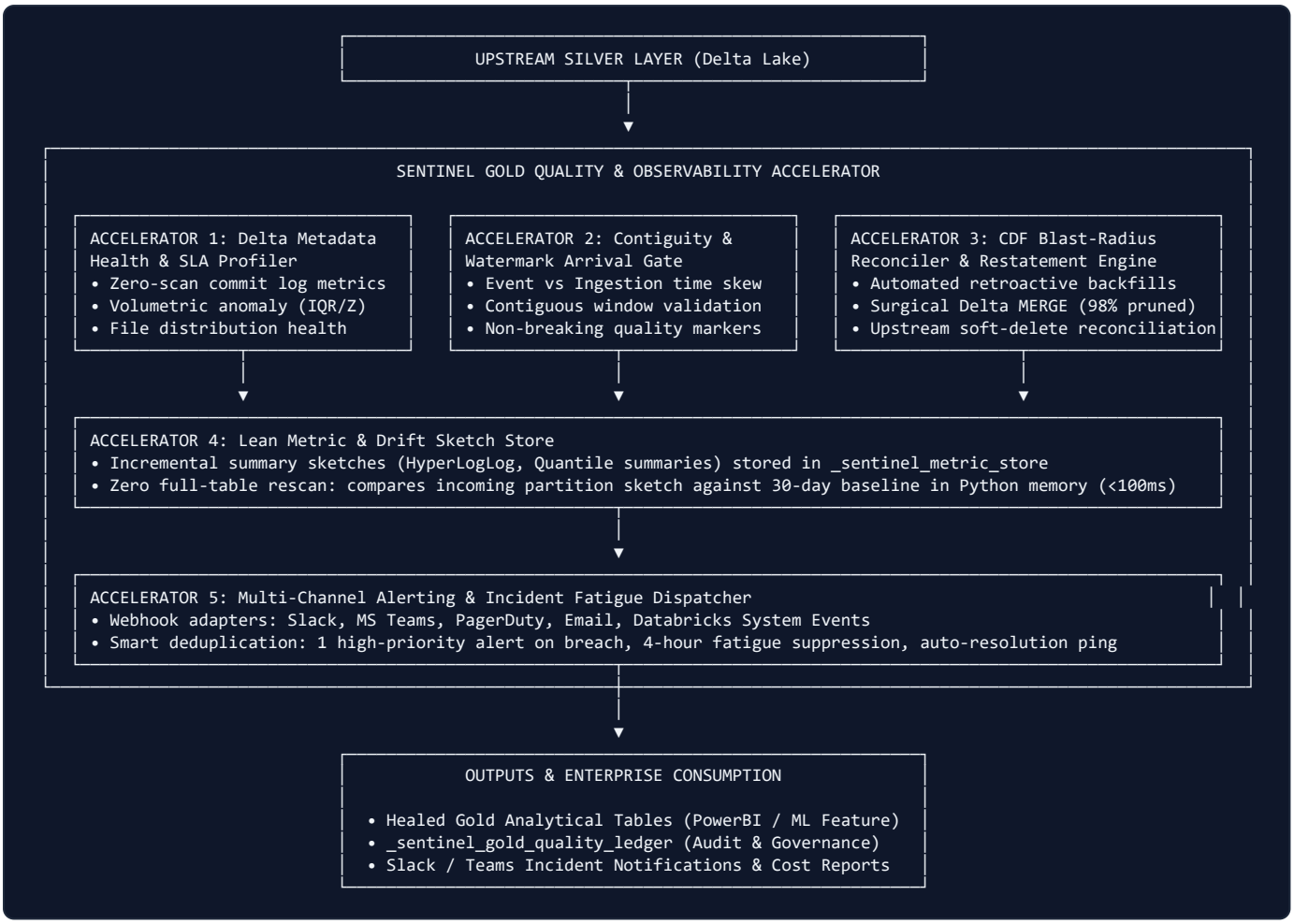
- **Late-Arriving & Missing Data**: Transactional records landing hours or days past the scheduled ingestion window corrupt rolling metrics, time-series trends, and ML feature stores.
- **The Vendor Cost Trap**: Third-party Databricks Marketplace offerings (Monte Carlo, Anomalo, Bigeye) and native Databricks Lakehouse Monitoring create massive cost overhead. They spin up dedicated serverless/classic clusters running heavy daily statistical profiling across petabytes of static historical data, charging $25k–$150k/year in SaaS licenses plus steep DBU markups.
- **Lack of Remediation**: Marketplace monitors fire alert webhooks when data is late, but they do not remediate the failure. Data engineers are still left manually writing slow, expensive full-table backfills.

The Sentinel Philosophy: Zero-Overhead, Native, Plug-and-Play

We propose an expanded suite of **modular, domain-agnostic accelerators** built entirely with native Python, SQL, and PySpark.

- **Zero Heavy Scans**: Leverages Delta Lake transaction logs, partition metadata, and embedded column statistics (min/max/null counts) instead of reading Parquet data files.
- **Automated Healing**: Detects delayed data, computes the exact blast radius, and executes targeted Delta MERGE partition pruning.
- **Universal Plug-and-Play**: Any Databricks project (retail, fintech, healthcare, IoT, adtech) can plug in via a declarative YAML/JSON contract with zero boilerplate code.

2. Accelerator Suite Architecture



3. Detailed Accelerator Breakdown

Accelerator 1: Zero-Scan Delta Metadata & Volumetric Profiler (`DeltaMetadataProfiler`)

- **Objective**: Replace heavy daily full-table statistical profiling with sub-second Delta log metadata inspection.
- **How It Works**:
  1. Inspects `DeltaTable.forPath().history(N)` and commit snapshots.
  2. Extracts partition sizes, file counts, and row counts from Delta log add-file actions (`stats` JSON containing `numRecords` , `minValues` , `maxValues` , `nullCount`).
  3. Computes moving averages and volumetric anomaly detection using statistical Z-Score and Interquartile Range (IQR) calculated entirely on lightweight metadata in Python memory.
  4. Flags sudden pipeline drops (e.g. 80% volume drop indicating upstream API ingestion outage) or spikes (e.g. 400% volume surge indicating duplicate batch ingestion).
- **Cost Advantage**: Runs on a single driver core in ~200ms. Reads ~20KB of metadata vs scanning 500GB+ of Parquet files (99.9% DBU savings).

Accelerator 2: Contiguity & Watermark Arrival Gate (`WatermarkArrivalGate`)

- **Objective**: Prevent incomplete rolling windows from publishing corrupted aggregates to executive dashboards or ML feature stores.
- **How It Works**:
  1. Evaluates incoming partition intervals against target rolling windows $[t - W + 1, t]$.

2. Tracks event-time watermarks versus ingestion-time watermarks to quantify upstream pipeline latency skew.
3. Evaluates declarative gating policies:
    - `FAIL_FAST`: Immediately halts the pipeline before writing corrupted data.
    - `DEGRADED_WITH_MARKER`: Computes Gold aggregations but appends non-breaking audit columns (`_dq_is_complete = False`, `_dq_completeness_score = 0.857`, `_dq_missing_intervals = ['2026-10-28']`). Downstream BI reports can flag or filter out unfinalized intervals without query breakage.
    - `QUARANTINE_WINDOW`: Diverts partial windows into a staged quarantine table until missing partitions land.

Accelerator 3: Surgical CDF Blast-Radius Reconciler (`CdfBlastReconciler`)

- **Objective**: Automate the backfill and restatement of historical Gold rolling windows when late-arriving data lands, eliminating manual backfills.
- **How It Works**:
    1. Inspects Delta Change Data Feed (CDF) (`table_changes`) on the Silver table to detect commits affecting past partition dates t_{late}.
    2. Calculates the exact **Blast Radius**: for a rolling window of size W, the set of affected Gold target dates is bounded by $[t_{\text{late}}, t_{\text{late}} + W - 1]$.
    3. Generates a targeted, partition-pruned Delta `MERGE` query that recomputes only the affected slice.
    4. Supports multi-entity primary keys (`[order_date, customer_id, store_id]`) to support wide dimensional schemas and prevent duplicate row match runtime exceptions.
    5. Computes and logs DBU scan reduction metrics (e.g., recomputing 7 days instead of 365 days yields a 98.1% scan cost reduction).

Accelerator 4: Lean Metric & Drift Sketch Store (`LeanDriftMonitor`)

- **Objective**: Detect statistical distribution drift in Gold KPIs without running heavy ML distribution profiling algorithms on full tables.
- **How It Works**:
    1. During normal scheduled Gold runs, computes lightweight statistical summaries (HyperLogLog distinct count sketches, min, max, mean, variance, null fraction) per partition.
    2. Appends summaries to an internal metadata Delta table: `_sentinel_metric_store`.
    3. When validating a new partition, compares its metric sketch against a 30-day moving baseline in Python memory.
    4. Flags KPI metric drift (e.g., average order value shifts > 3 standard deviations or null rate jumps from 0.01% to 5.0%) without ever scanning past historical Parquet data.

Accelerator 5: Multi-Channel Alerting & Incident Fatigue Dispatcher (`AlertDispatcher`)

- **Objective**: Deliver actionable, non-fatiguing incident alerts with zero external SaaS dependencies.
- **How It Works**:
    1. Pluggable notification channels:
        - **Slack Webhook**: High-contrast Block Kit alert cards.
        - **Microsoft Teams**: Adaptive Card payloads.
        - **PagerDuty**: Events API v2 for critical SLA breaches.
        - **Unity Catalog System Table**: Writes to `_sentinel_gold_quality_ledger`.
    2. **Alert Fatigue Prevention Engine**:
        - Deduplicates recurring failure pings within a configurable cooldown window (e.g., 4 hours).
        - Maintains incident state: `TRIGGERED`, `ACKNOWLEDGED`, `AUTO_RESOLVED`.

- Automatically sends a resolution notification once missing partitions arrive and blast-radius reconciliation succeeds.

3. Actionable Alert Payload:
   - Includes: Table Name, Failed Date, Missing Intervals, Blast Radius Span, Estimated DBU Savings, and the exact generated Delta `MERGE` query.

Accelerator 6: Universal Plug-and-Play Contract Engine (`SentinelConfigEngine`)

- **Objective**: Enable any lakehouse project to adopt the framework in minutes with zero custom pipeline boilerplate.
- **How It Works**:
  1. Projects define their quality and SLA contracts in a clean YAML file (`sentinel_contracts.yml`).
  2. The Sentinel framework auto-discovers all table contracts, binds the SLA gates, contiguity validators, reconcilers, and alert webhooks, and executes as a standard Databricks Job task or Lakeflow step.

---

4. Plug-and-Play Configuration Specification

Any lakehouse team can drop the following declarative specification into their project repository:

```yaml
version: "1.0"
lakehouse:
  catalog: "main"
  schema: "ecommerce"
  audit_schema: "governance"

alerts:
  slack:
    webhook_secret: "sentinel/slack_webhook"
    channel: "#data-platform-alerts"
  cooldown_hours: 4

tables:
  - target_table: "gold_rolling_revenue"
    source_table: "silver_orders"
    partition_column: "order_date"
    time_column: "order_timestamp"
    primary_keys: ["order_date", "region_id"]
    cadence: "DAILY"
    sla:
      grace_period_hours: 4
      min_expected_rows: 500
      volume_anomaly_threshold_pct: 40.0
    gating_policy: "DEGRADED_WITH_MARKER"
    rolling_windows:
      - name: "7d_revenue"
        window_size_days: 7
        metric_expr: "sum(total_amount)"
      - name: "30d_orders"
        window_size_days: 30
        metric_expr: "count(order_id)"
    reconciliation:
      auto_heal_late_arrivals: true
      max_lookback_days: 30
    monitoring:
      drift_detection: true
      tracked_columns: ["total_amount", "discount_amount"]
```

---

5. Cost & Compute Benchmark Comparison

Dimension	Databricks Marketplace / SaaS (Monte Carlo, Anomalo)	Databricks Lakehouse Monitoring	
SaaS Licensing	$25,000 to $150,000+ per year	$0 (included in Databricks)	**$0 (100% Native Open Source)**
Compute Consumption	Spawns external compute / heavy full-table profiling jobs	Runs recurring multi-node cluster profiles on all historical data	**Near-Zero**: Metadata-first inspection (<1MB read vs 500GB+ scan)
Late Data Resolution	Alert webhook only; engineers must manually backfill	Drift metric logged after the fact; no automated backfill	**Automated Blast-Radius Healing**: Surgically recomputes only $[t_{\text{late}}, t_{\text{late}} + W - 1]$ via Delta MERGE
Rolling Window Contiguity	Generic row count profiling; misses interval gaps	Generic time-series metrics	**Native Window Math**: Injects contiguity scores (k/W) and missing interval lists
Deployment Complexity	High (third-party agents, external network egress, secrets)	Medium (proprietary Databricks system tables & DLT requirements)	**Plug-and-Play**: 1 Python package / YAML contract; runs directly inside existing jobs

6. Phased Implementation Roadmap

Phase	Module	Scope & Deliverables
Phase 1	**Delta Metadata Profiler** (`sentinel.gold.profiler`)	Zero-scan partition row-count, file health, and volume anomaly (Z-score/IQR) detection using Delta commit logs
Phase 2	**Multi-Window Watermark Router** (`sentinel.gold.watermark`)	Ingestion vs event-time latency skew tracker and multi-policy window routing (`FAIL_FAST`, `DEGRADED`, `QUARANTINE`)
Phase 3	**CDF Blast-Radius Reconciler** (`sentinel.gold.reconciler`)	Full Change Data Feed integration for automated retroactive Gold table healing with multi-entity primary keys
Phase 4	**Lean Metric Drift Store** (`sentinel.gold.drift`)	Incremental HyperLogLog / statistical sketch engine and drift detector against 30-day baseline
Phase 5	**Alert Fatigue Dispatcher** (`sentinel.gold.alerts`)	Slack Block Kit & MS Teams webhook adapters with 4-hour alert deduplication and auto-resolution notifications
Phase 6	**Universal YAML Contract Engine** (`sentinel.gold.contract`)	YAML schema parser, CLI runner, and Databricks Asset Bundle (DAB) integration task